

Arbeitspaket: Backend-Entwicklung Heizöl-Vorabdisposition

1. Ziel des Arbeitspakets

Ziel dieses Arbeitspakets war die Entwicklung eines funktionsfähigen Backend-Prototyps für den Bestätigungsprozess einer Heizöl-Lieferung.

Das Backend übernimmt dabei die Rolle der Integrationsschicht zwischen DISPO-System, Kundenkommunikation und Prozesssteuerung. Es empfängt Bestätigungsanfragen vom DISPO-System, erzeugt einen Bestätigungslink, versendet diesen über den gewählten Kommunikationskanal, verarbeitet Kundenantworten und aktualisiert den Bestätigungsstatus. Zusätzlich wird über Camunda ein Timeout-Prozess abgebildet, falls der Kunde nicht rechtzeitig antwortet.

2. Fachlicher Umfang

Der entwickelte Backend-Prototyp deckt folgenden fachlichen Ablauf ab:

1. DISPO-System erstellt eine Bestätigungsanfrage.
 2. Backend validiert die übergebenen Liefer- und Kundendaten.
 3. Backend speichert einen Snapshot der Auftragsdaten.
 4. Backend erzeugt eine aktive Confirmation Request mit sicherem Token.
 5. Backend versendet den Bestätigungslink per E-Mail oder SMS.
 6. Kunde ruft die Bestätigungsseite über den Token auf.
 7. Kunde bestätigt oder lehnt das Lieferfenster ab.
 8. Backend speichert die Kundenantwort.
 9. Backend aktualisiert den Status auf `CONFIRMED`, `REJECTED` oder `NO_RESPONSE`.
 10. Backend sendet den aktualisierten Status per HTTP Callback an das DISPO-System.
 11. Camunda setzt nach Ablauf der Frist automatisch den Status `NO_RESPONSE`, falls keine Antwort eingegangen ist.
-

3. Technischer Umfang

3.1 REST API für DISPO

Implementiert wurde ein REST-Endpunkt für das DISPO-System:

```
POST /api/dispo/confirmation-requests
```

Der Endpunkt nimmt Lieferdaten, Kundendaten und den gewünschten Kommunikationskanal entgegen.

Unterstützte Kommunikationskanäle:

EMAIL
SMS

Wichtige umgesetzte Regeln:

- neue Anfrage erzeugt Status `SENT`
- unveränderte doppelte Anfrage erzeugt keine zweite Nachricht
- geänderte Anfrage invalidiert die vorherige aktive Anfrage
- Lieferzeitfenster wird validiert
- je nach Kommunikationskanal werden E-Mail-Adresse oder Telefonnummer benötigt

3.2 Customer API

Implementiert wurden Endpunkte für den Kunden:

```
GET /api/customer/confirmations/{token}  
POST /api/customer/confirmations/{token}/confirm  
POST /api/customer/confirmations/{token}/reject
```

Die Customer API prüft:

- ob der Token existiert
- ob die Anfrage noch aktiv ist
- ob die Anfrage nicht abgelaufen ist
- ob bereits eine Antwort gespeichert wurde

3.3 Persistenzschicht

Umgesetzt wurde die persistente Speicherung mit PostgreSQL, Spring Data JPA und Flyway.

Wesentliche Tabellen:

```
order_snapshot  
confirmation_request  
customer_response
```

Zusätzlich wurde eine Migration für den Kommunikationskanal ergänzt.

3.4 E-Mail-Versand

Der E-Mail-Versand wurde über Spring Mail und Thymeleaf umgesetzt.

Für die lokale Entwicklung wird Mailpit verwendet.

Umgesetzte Punkte:

- HTML-Mail-Template
 - Link zur Frontend-Route `/confirmation/{token}`
 - lokale SMTP-Konfiguration über Mailpit
 - Fehlerbehandlung bei E-Mail-Versandfehlern
-

3.5 SMS-Versand

Für SMS wurde ein abstrahierter Versandmechanismus umgesetzt.

Da im MVP kein echter SMS-Provider angebunden wird, wurde ein eigener `sms-mock` erstellt.

Umgesetzte Punkte:

- Kommunikationskanal `SMS`
 - Telefonnummer im DISPO Request
 - SMS-Versand über HTTP an einen Mock-Service
 - SMS enthält denselben Bestätigungslink wie die E-Mail
 - lokale Kontrolle der verschickten SMS über SMS Mock
-

3.6 DISPO Callback

Nach einer Kundenantwort oder nach Ablauf des Timeouts sendet das Backend den aktuellen Status zurück an das DISPO-System.

Umgesetzt wurde ein echter HTTP-Client auf Basis von Spring `RestClient`.

Callback-Inhalt:

```
{
  "externalOrderId": "A-3001",
  "confirmationStatus": "CONFIRMED",
  "customerComment": "Optionaler Kommentar"
}
```

Zusätzlich wurde ein `dispo-mock` erstellt, der die Callbacks lokal entgegennimmt und protokolliert.

3.7 Camunda Workflow

Camunda wird verwendet, um den Timeout-Fall abzubilden.

Umgesetzt wurden zwei Prozessbereiche:

1. Timeout-Prozess für ausbleibende Kundenantwort

2. Callback-Prozess für Statusupdates an DISPO

Fachliche Logik:

- nach Erstellen einer Confirmation Request wird ein Camunda-Prozess gestartet
- der Prozess wartet bis zur konfigurierten Deadline
- wenn bis dahin keine Kundenantwort existiert, wird der Status auf `NO_RESPONSE` gesetzt
- die Anfrage wird deaktiviert
- anschließend wird ein DISPO Callback ausgelöst

3.8 Fehlerbehandlung

Es wurde ein zentraler `GlobalExceptionHandler` umgesetzt.

Behandelte Fehlerarten:

- Validierungsfehler
- ungültiges Lieferfenster
- ungültiger oder unbekannter Token
- abgelaufene Anfrage
- inaktive Anfrage
- doppelte Kundenantwort
- E-Mail-Versandfehler
- SMS-Versandfehler
- nicht gefundene Ressourcen
- unerwartete technische Fehler

Fehler werden einheitlich als JSON zurückgegeben.

3.9 Konfiguration und Profile

Die Konfiguration wurde in mehrere Profile aufgeteilt:

```
application.yml
application-dev.yml
application-prod.yml
```

Ziel:

- Basiskonfiguration zentral halten
- lokale Entwicklungswerte in `dev`
- produktionsähnliche Werte in `prod`
- sensible oder umgebungsspezifische Werte leichter austauschbar machen

3.10 Docker-basierte lokale Infrastruktur

Für die lokale Entwicklung wurde Docker Compose vorbereitet.

Enthaltene Services:

```
PostgreSQL
Mailpit
pgAdmin
DISPO Mock
SMS Mock
```

Das Backend selbst wird während der Entwicklung lokal aus IntelliJ oder über Maven gestartet.

3.11 Swagger / OpenAPI

Zur Unterstützung der Frontend-Entwicklung wurde Swagger/OpenAPI eingebunden.

Verfügbare URLs:

```
http://localhost:8080/swagger-ui.html
http://localhost:8080/v3/api-docs
```

Damit können Frontend-Entwickler die REST-Schnittstellen einsehen und testen.

3.12 Integrationstests

Es wurden Integrationstests für die wichtigsten Backend-Flows umgesetzt.

Getestete Bereiche:

- Erstellung einer DISPO Confirmation Request
- Duplicate Handling
- Customer Preview
- Customer Confirm
- Customer Reject
- Speicherung der Kundenantwort
- Timeout auf `NO_RESPONSE`
- Camunda-Prozessverhalten
- DISPO Callback
- Fehlerfälle
- E-Mail/SMS-bezogene Validierung

Verwendete Werkzeuge:

JUnit 5
Spring Boot Test
MockMvc
Testcontainers PostgreSQL
MockitoBean
Awaitility

4. Ergebnis des Arbeitspakets

Das Ergebnis ist ein lauffähiger Backend-Prototyp mit vollständigem End-to-End-Ablauf.

Der Prototyp kann lokal gestartet und demonstriert werden:

DISPO Request
→ Backend
→ E-Mail oder SMS
→ Kunde bestätigt oder lehnt ab
→ Backend speichert Antwort
→ DISPO erhält Callback

Zusätzlich ist der Timeout-Fall über Camunda abgebildet:

DISPO Request
→ keine Kundenantwort
→ Camunda Timer läuft ab
→ Status wird NO_RESPONSE
→ DISPO erhält Callback

5. Realistische Aufwandsschätzung für die bisher umgesetzte Backend-Arbeit

Die folgende Schätzung beschreibt nicht den vollständigen Projektaufwand von 300 Stunden, sondern den realistischen Aufwand für die bisher umgesetzten Backend-Aufgaben. Die Werte enthalten Analyse, Implementierung, manuelles Testen, Debugging, kleinere Korrekturen und Dokumentation.

Nr.	Tätigkeit	Realistischer Aufwand
1	Fachlichen Ablauf DISPO → Backend → Kunde → DISPO verstehen und grob strukturieren	4 h
2	REST-Schnittstellen für DISPO und Customer API entwerfen	4 h

Nr.	Tätigkeit	Realistischer Aufwand
3	Spring-Boot-Projektstruktur, Packages und Grundkonfiguration aufbauen	3 h
4	Datenmodell mit <code>order_snapshot</code> , <code>confirmation_request</code> , <code>customer_response</code> umsetzen	6 h
5	Flyway-Migrationen für Datenbankschema erstellen und anpassen	3 h
6	DISPO Endpoint <code>POST /api/dispo/confirmation-requests</code> implementieren	5 h
7	Businesslogik für neue Requests, Status <code>SENT</code> und Speicherung der Auftragsdaten	6 h
8	Duplicate Handling und Invalidierung alter aktiver Requests	5 h
9	Token-Erzeugung und Token-basierter Zugriff	3 h
10	Customer Preview API implementieren	3 h
11	Customer Confirm/Reject API implementieren	5 h
12	Kundenantwort mit Kommentar speichern	3 h
13	Zentrales Error Handling und einheitliches Fehlerformat	5 h
14	E-Mail-Versand mit Spring Mail und Mailpit integrieren	5 h
15	E-Mail-Template und Link-Erzeugung umsetzen	3 h
16	Camunda Timeout-Prozess für <code>NO_RESPONSE</code> integrieren	8 h
17	Camunda Callback-Prozess für DISPO Statusupdates integrieren	5 h
18	DISPO Callback per HTTP Client implementieren	4 h
19	DISPO Mock Service für lokale Tests erstellen	5 h
20	Kommunikationskanal <code>EMAIL</code> / <code>SMS</code> in Request, Entity und Validierung einbauen	6 h
21	SMS-Versand-Abstraktion und SMS Mock Client implementieren	5 h
22	SMS Mock Service für lokale Tests erstellen	5 h
23	Docker Compose für PostgreSQL, Mailpit, pgAdmin, DISPO Mock und SMS Mock anpassen	5 h
24	Profile <code>dev</code> / <code>prod</code> und Konfiguration aufräumen	4 h
25	Swagger/OpenAPI einbinden und kontrollieren	2 h
26	Integrationstests für DISPO API schreiben	6 h
27	Integrationstests für Customer API schreiben	6 h
28	Integrationstests für Camunda Timeout und Callback schreiben	8 h

Nr.	Tätigkeit	Realistischer Aufwand
29	Integrationstests für E-Mail/SMS-Validierung und Kommunikationskanäle ergänzen	5 h
30	Manuelles End-to-End-Testen mit Mailpit, SMS Mock, DISPO Mock und pgAdmin	6 h
31	Debugging von Camunda Jobs, SQL, Profilen und lokalen Docker-Problemen	7 h
32	README und technische Entwicklerdokumentation aktualisieren	5 h
33	Vorbereitung eines Demo-Flows für Präsentation / Abnahme	3 h

Realistische Gesamtschätzung für die bisherige Backend-Arbeit: ca. 160 h

Diese Zahl ist als realistische technische Aufwandsschätzung für die bisher erstellte Backend-Funktionalität zu verstehen. Sie entspricht nicht dem vollständigen Projektumfang und enthält keine Frontend-Arbeit.

6. Kompakte Übersicht des Backend-Anteils bis zum aktuellen Stand

Bereich	Aufwand
Analyse, Schnittstellen und Architekturentscheidungen	12 h
DISPO API, Datenmodell und Persistenz	23 h
Customer API und Token-basierter Antwortfluss	14 h
E-Mail-Versand und Mailpit-Integration	8 h
Camunda Timeout und Callback-Prozesse	20 h
DISPO HTTP Callback und DISPO Mock	9 h
SMS-Kanal und SMS Mock	16 h
Docker Compose, Profile, Swagger und lokale Infrastruktur	11 h
Integrationstests und manuelle End-to-End-Tests	31 h
Debugging, Refactoring und Dokumentation	16 h

Gesamt: ca. 160 h

7. Abgrenzung

Nicht Bestandteil dieses Arbeitspakets waren:

- Implementierung des Frontends
- echte produktive DISPO-Anbindung
- echter SMS-Provider
- echter SMTP-Provider
- Authentifizierung und Autorisierung
- produktionsreifes Deployment in Kubernetes
- vollständiges Monitoring und Observability

Diese Punkte sind entweder Mock-Komponenten im MVP oder spätere Erweiterungen.

8. Aktuelle MVP-Limitierungen

Der aktuelle Stand ist ein funktionsfähiger Prototyp, aber noch kein produktionsreifes System.

Bekannte Einschränkungen:

- SMS-Versand ist gemockt
 - DISPO-System ist gemockt
 - SMTP läuft lokal über Mailpit
 - Tokens werden im MVP noch im Klartext gespeichert
 - Retry/Outbox-Mechanismus ist nur teilweise über Camunda Job Retry abgebildet
 - keine Authentifizierung
 - keine produktive Secrets-Verwaltung
 - keine Kubernetes-Manifeste
-

9. Fazit

Das Backend-Arbeitspaket bildet den zentralen technischen Ablauf des Projekts ab. Es stellt die Verbindung zwischen DISPO, Kundenkommunikation, Statusverwaltung und Prozessautomatisierung her.

Für den MVP ist der Backend-Teil damit weitgehend vollständig. Die verbleibenden Aufgaben liegen vor allem in Stabilisierung, Dokumentation, Präsentationsvorbereitung und späterer produktionsnaher Erweiterung.